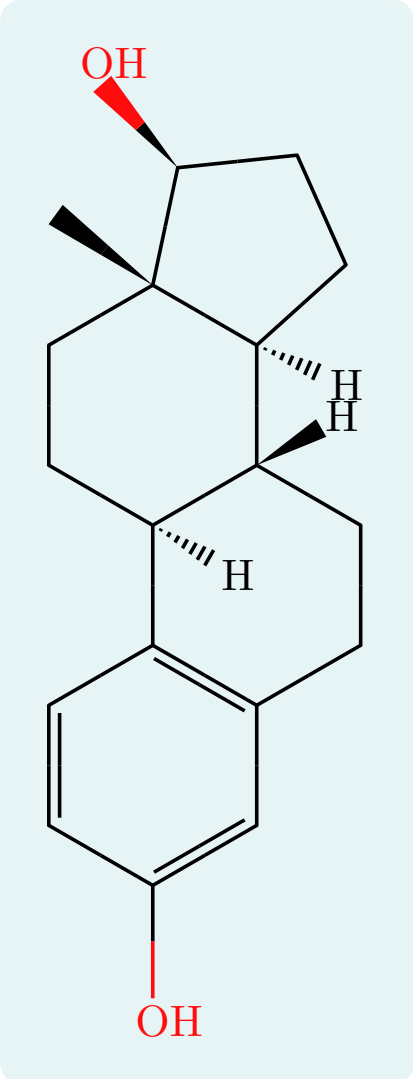
typed-smiles

Render SMILES strings as 2D molecular diagrams

User Guide



Version 0.5.0

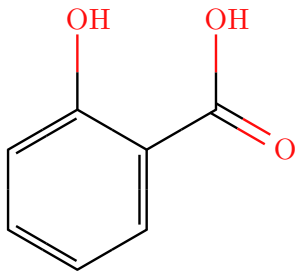
Contents

1 Introduction

typed-smiles renders SMILES strings as 2D skeletal molecular diagrams inside Typst documents. A bundled WebAssembly plugin parses the SMILES, computes atom coordinates, implicit hydrogens, and stereochemistry, and the Typst layer draws the bonds, atom labels, colors, charges, hydrogens, wedges, and reaction helpers.

This guide documents every function, argument, and package-specific syntax extension. Each feature comes with a runnable example: the source is on the left, its rendered output on the right.

```typst
#smiles("OC1=CC=CC=C1C(=O)O")
```



2 Getting started

2.1 Installation and import

Import the package from the Typst preview namespace. A wildcard import gives you every public symbol:

```
#import "@preview/typed-smiles:0.5.0": *
```
Typst

Or import only what you need:

```
#import "@preview/typed-smiles:0.5.0": smiles, ce, mol, rxn-arrow, reaction
```
Typst

The package exports five symbols:

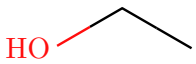
Symbol	Purpose
`smiles`	Render a SMILES string as a molecule (the main function).
`ce`	Chemical formulas and equations, re-exported from `chemformula`.
`mol`	Wrap a molecule with an optional caption.
`rxn-arrow`	A reaction arrow with conditions above and below.
`reaction`	Lay out a multi-step reaction scheme.

2.2 Your first molecule

Call `smiles` with a SMILES string.

```
Ethanol: #smiles("CCO")
```
Typst

Ethanol:



3 The `smiles()` function

`smiles` is the main function. Its full signature is:

```typst
#smiles(
  smiles-str,
  scale: 1.0,
  bond-length: none,
  font-size: none,
  font: "New Computer Modern",
  bond-stroke: none,
  color: true,
  rotation: 0deg,
  mirror: none,
  show-all-h: false,
  lone-pairs: none,
  atom-colors: (:),
  show-indices: false,
)
```

3.1 Argument reference

Argument	Type	Default	Description
`smiles-str`	`str`	(required)	The SMILES string.
`scale`	`float`	`1.0`	Balanced scale for bond length, label size, and stroke at once.
`bond-length`	`float` / `none`	`none`	Bond length factor (`1.0` is 30 pt). Overrides `scale` for length.
`font-size`	`length` / `none`	`none`	Atom-label font size. Overrides `scale` for labels.
`font`	`str`	`"New Computer Modern"`	Font family for atom labels.
`bond-stroke`	`length` / `none`	`none`	Bond stroke width. Overrides `scale` for strokes.
`color`	`bool`	`true`	Apply Jmol CPK atom colors.
`rotation`	`angle`	`0deg`	Rotate the molecule; labels stay upright.
`mirror`	`none` / `"horizontal"` / `"vertical"`	`none`	Reflect the molecule across an axis; ap-

			plied before `rotation`. Wedges and hashes are exchanged to preserve the depicted configuration.
`show-all-h`	`bool`	`false`	Also label carbon implicit hydrogens.
`lone-pairs`	`none` / `"dots"` / `"lines"`	`none`	Draw optional nonbonding electron pairs on skeletal atom labels.
`atom-colors`	`dictionary`	`(:)`	Color overrides for elements and labels. Element-symbol keys (e.g. `O: red`) override CPK colors; brace-quoted label keys (e.g. `"{PPh3}": purple`) override a specific abbreviated group. See Section 7.

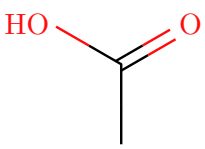
Note. `scale` sizes everything together. Use `bond-length`, `font-size`, and `bond-stroke` to override one dimension on its own; each defaults to `none`, meaning it follows `scale`.

3.2 `smiles-str`

The positional argument is the SMILES string.

```
1  #smiles("CC(=O)O")
```

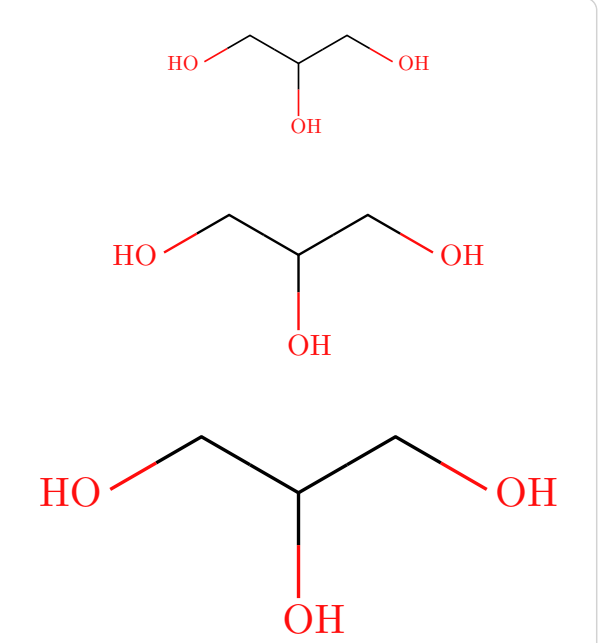
Typst



3.3 scale

Scales bond length, labels, and stroke proportionally.

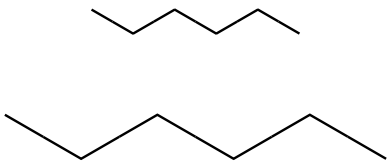
```typst
#smiles("OCC(O)CO", scale: 0.7) \
#smiles("OCC(O)CO", scale: 1.0) \
#smiles("OCC(O)CO", scale: 1.4)
```



3.4 bond-length

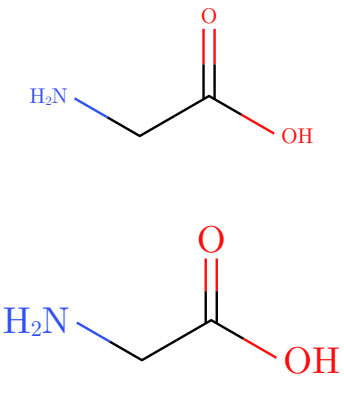
Sets bond length on its own (`1.0` is 30 pt per bond), leaving label size and stroke under `scale`.

```typst
#smiles("CCCCCC", bond-length: 0.6) \
#smiles("CCCCCC", bond-length: 1.1)
```



3.5 font-size

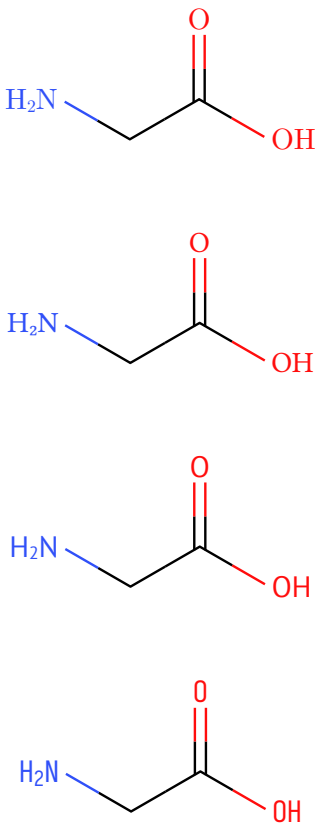
```typst
#smiles("OC(=O)CN", font-size: 8pt) \
#smiles("OC(=O)CN", font-size: 14pt)
```



3.6 font

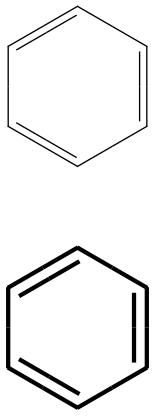
Match the document typeface, or pick something with character.

```typst
#smiles("OC(=O)CN", font: "New Computer Modern") \
#smiles("OC(=O)CN", font: "Libertinus Serif") \
#smiles("OC(=O)CN", font: "PT Sans") \
#smiles("OC(=O)CN", font: "Iosevka")
```



3.7 bond-stroke

```typst
#smiles("C1=CC=CC=C1", bond-stroke: 0.5pt) \
#smiles("C1=CC=CC=C1", bond-stroke: 1.6pt)
```

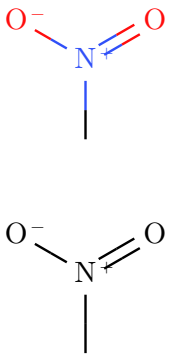


3.8 color

CPK colors are on by default (see Section 7). Set `color: false` for a monochrome diagram.

```
1  #smiles("C[N+](=O)[O-]") \
2  #smiles("C[N+](=O)[O-]", color: false)
```

Typst

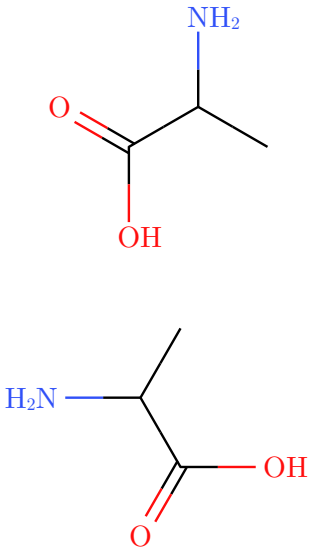


3.9 rotation

Rotates the whole molecule while keeping every label upright.

```
1  #smiles("CC(N)C(=O)O", rotation:
   0deg) \
2  #smiles("CC(N)C(=O)O", rotation: 90deg)
```

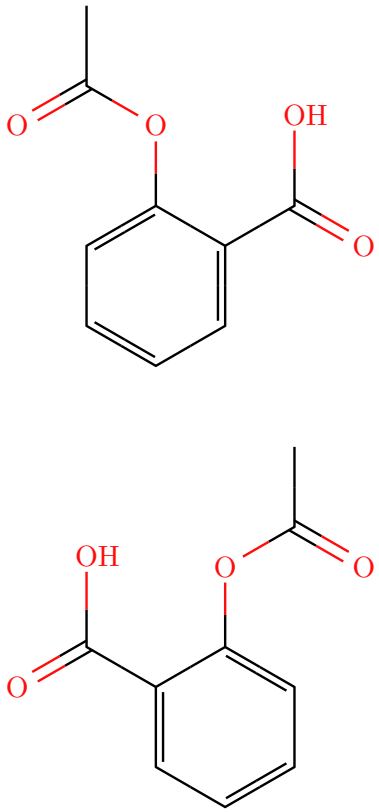
Typst



3.10 `mirror`

Reflects the molecule across an axis — something no `rotation` can do, since a rotated drawing is never its own mirror image. `"horizontal"` swaps left and right, `"vertical"` swaps top and bottom; mirroring is applied before `rotation`. Useful to face a reacting group toward a reaction arrow. Inside `reaction()`, pass it per molecule: `mol("OC1CCCCC1", mirror: "horizontal")`.

```typst
#smiles("CC(=O)OC1=CC=CC=C1C(=O)O")
\
#smiles(
  "CC(=O)OC1=CC=CC=C1C(=O)O",
  mirror: "horizontal",
)
```

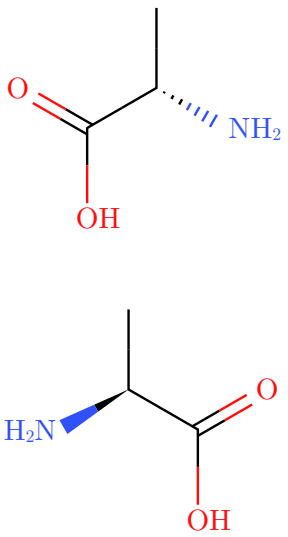


Note. Mirroring exchanges wedges and hashes: a reflected 2D skeleton with unchanged wedges would depict the enantiomer, while reflecting and swapping is equivalent to turning the molecule over — the depicted configuration at every stereocenter is preserved.

A stereocenter keeps its configuration when mirrored.

```
#smiles("N[C@@H](C)C(=O)O") \
#smiles(
  "N[C@@H](C)C(=O)O",
  mirror: "horizontal",
)
```

Typst

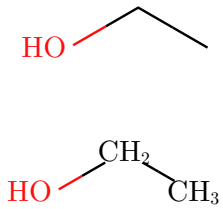


3.11 show-all-h

Hydrogens on heteroatoms are shown by default; carbon hydrogens are hidden. Set `show-all-h: true` to label every implicit hydrogen.

```
#smiles("CCO") \
#smiles("CCO", show-all-h: true)
```

Typst

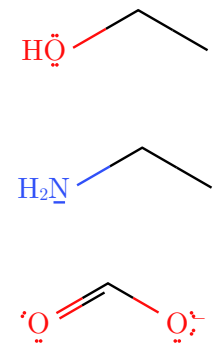


3.12 lone-pairs

Set `lone-pairs` to `"dots"` or `"lines"` to annotate the skeletal drawing with non-bonding electron pairs on common organic heteroatoms and charged atoms. The default `none` keeps lone pairs hidden.

```
#smiles("CCO", lone-pairs:
"dots") \
#smiles("CCN", lone-pairs: "lines") \
#smiles("[O-]C=O", lone-pairs: "dots")
```

Typst



Note. Lone pairs are inferred for a conservative organic subset such as N, O, S, P, halogens, and simple charged forms. They are an annotation on the skeletal structure; the carbon skeleton and implicit-hydrogen conventions do not change. For example, ammonium (`[NH4+]`) has no nitrogen

lone pair. On terminal hydrogen-bearing labels such as `OH`, `OH-`, and `NH2`, lone pairs are placed from the rendered heavy-atom glyph center. For charged bare atoms such as `[O-]` or `[Br-]`, the charge mark does not move the atom center used for lone pairs.

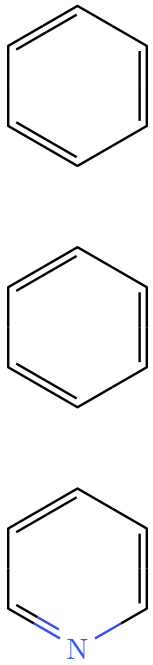
4 Atoms and charges

Standard SMILES syntax (atoms, bonds, branches, ring closures) is supported. This section covers the points specific to typed-smiles.

4.1 Aromatic rings

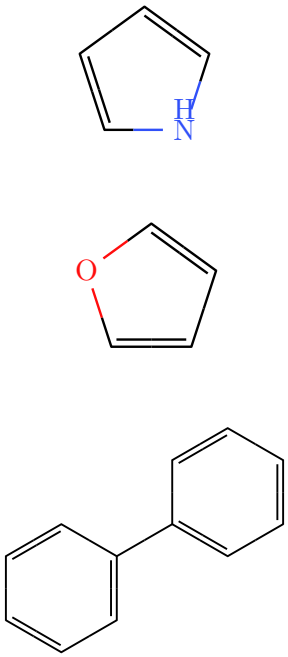
Aromatic rings can be written in either OpenSMILES form: lowercase aromatic atoms (`c1ccccc1`) or Kekulé notation with explicit alternating double bonds (`C1=CC=CC=C1`). Aromatic input is kekulized on parse, so both render identically.

```
#smiles("c1ccccc1") \
#smiles("C1=CC=CC=C1") \
#smiles("c1ccncc1")
```



Bracket aromatic atoms carry their hydrogen count explicitly, as in pyrrole's `[nH]`. A single bond between two aromatic rings (biphenyl) may be written with or without the explicit `-`.

```typst
#smiles("c1cc[nH]c1") \
#smiles("c1occc1") \
#smiles("c1ccccc1-c1ccccc1", scale: 0.8)
```



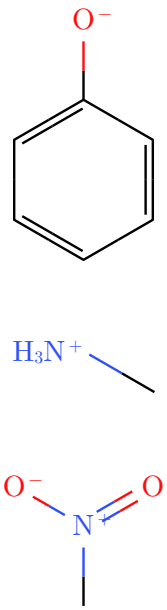
Note. Kekulization does not renumber anything: atom indices follow SMILES writing order for aromatic input exactly as for Kekulé input, so `show-indices`, `highlight(...)`, and `arrow(...)` references work unchanged, including on ring bonds whose double bond only appears after kekulization. A ring that cannot be kekulized (for example pyrrole written `c1ccnc1`, without the `[nH]` hydrogen) or an aromatic atom outside a ring is reported as an error.

4.2 Charges

Formal charges inside brackets render as a raised sign after the atom. Hydrogen counts and charge marks use the atom-label size so they remain legible when labels are scaled.

```
#smiles("[O-]C1=CC=CC=C1") \
#smiles("C[NH3+]") \
#smiles("C[N+](=O)[O-]")
```

Typst



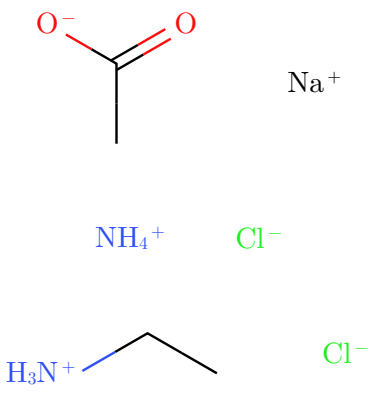
Note. A bracketed element such as `[N]` is a nitrogen atom. To print an arbitrary text label instead, use the abbreviation syntax `{N}` (see Section 8).

4.3 Salts and disconnected structures

A dot (`.`) means "no bond": it separates disconnected fragments, as in salts, counterions, and hydrates. Each fragment is laid out on its own and the fragments are arranged left to right in writing order.

```
#smiles("CC(=O)[O-].[Na+]") \
#smiles("[NH4+].[Cl-]") \
#smiles("CC[NH3+].[Cl-]")
```

Typst



Note. Atom indices stay global across fragments, still following SMILES writing order, so `show-indices`, `highlight(...)`, and `arrow(...)` can reference atoms in any fragment of the same `smiles()` call. A ring closure written across a dot (as in the OpenSMILES example `C1.C1`, ethane) still forms its bond.

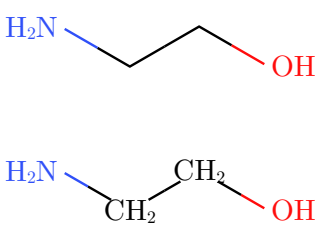
5 Hydrogens

5.1 Default display

Heteroatom hydrogens are shown by default; carbon hydrogens are hidden for a clean skeleton. `show-all-h: true` labels carbon hydrogens as well.

```
#smiles("OCCN") \
#smiles("OCCN", show-all-h: true)
```

Typst

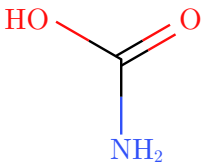


5.2 Explicit bracket hydrogens

A hydrogen written as its own bracket atom (for example the `[H]` atoms in `C([H])([H])[H]`) is folded into its neighbor's hydrogen count, exactly like an implicit hydrogen. Carbon-bound hydrogens stay hidden and heteroatom hydrogens are still labeled, so a fully hydrogen-suppressed SMILES depicts the same as its skeletal form. Hydrogen counts written inside a single bracket, such as `[OH-]` or `[NH4+]`, are unaffected and still shown.

```
#smiles("C([H])([H])([H])[H]") \
#smiles("N([H])([H])C(=O)O")
```

Typst

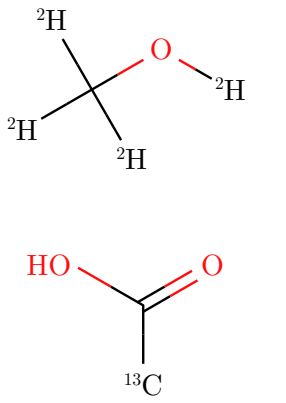


5.3 Isotopes

A bracket isotope is drawn as a leading superscript mass number. Isotopically labeled hydrogens such as deuterium `[2H]` are kept as drawn atoms rather than folded away.

```
#smiles("[2H]OC([2H])([2H])[2H]")
\
#smiles("[13CH3]C(=O)O")
```

Typst

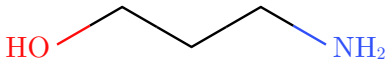


5.4 Label orientation

Terminal heteroatom labels orient so the bond meets the heavy atom rather than the trailing hydrogen, at any angle.

```
1  #smiles("NCCCO")
```

Typst



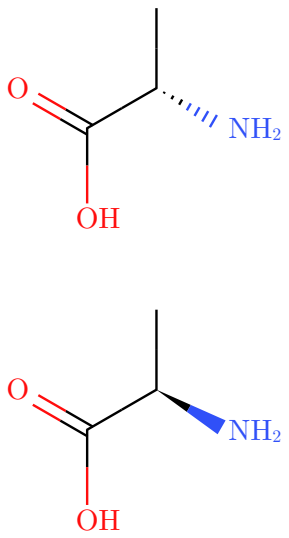
6 Stereochemistry

typed-smiles supports both standard SMILES stereo notations, plus a manual override for drawing wedges directly.

6.1 Tetrahedral centers: `@` and `@@`

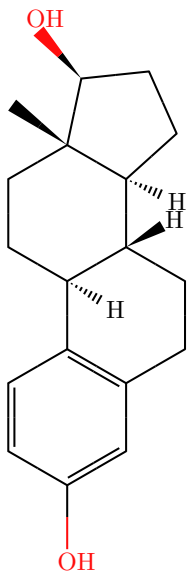
A bracket atom carrying `@` or `@@` becomes a wedge (toward the viewer) or hashed bond (away), computed from the 2D layout so the depiction matches the SMILES in the conventional orientation.

```typst
#smiles("N[C@@H](C)C(=O)O") \
#smiles("N[C@H](C)C(=O)O")
```



The renderer wedges an exocyclic substituent (such as OH or CH_3) where one exists, and an explicit hydrogen otherwise. Fused systems work too.

```typst
#smiles(
  "C[C@]12CC[C@H]3[C@H]
  ([C@@H]1CC[C@@H]2O)CCC4=C3C=CC(=C4)O",
  scale: 0.85,
)
```

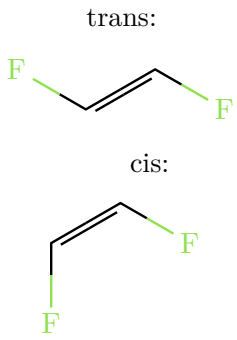


Note. The geometry is drawn correctly, but R/S descriptors are not computed. Ring stereochemistry between adjacent centers and bridged bicyclics may need a manual adjustment (see Section 14).

6.2 Double bonds: / and \

Directional bonds `/` and `\` around a double bond set its cis/trans geometry. They come in pairs, one on each side of the `=`.

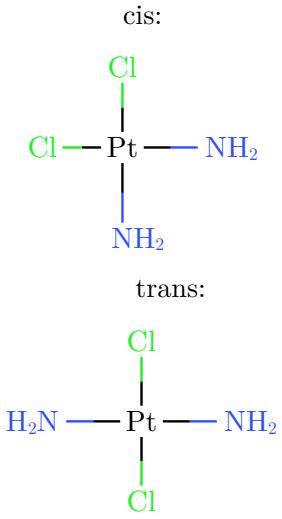
```typst
trans: #smiles("F/C=C/F")
#h(2em)
cis: #smiles("F/C=C\F")
```



6.3 Square-planar centers: @SP1, @SP2, @SP3

Square-planar stereochemistry is depicted exactly: the four neighbors sit at 90° around the center, arranged so the line traced through them in SMILES order forms the class's shape — 'U' for `@SP1`, '4' for `@SP2`, 'Z' for `@SP3`. The classic example is cis- versus trans-platin.

```typst
cis: #smiles("N[Pt@SP1](N)(Cl)Cl")
#h(2em)
trans: #smiles("N[Pt@SP2](N)(Cl)Cl")
```



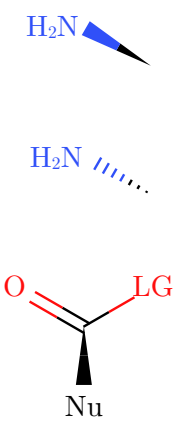
Note. Trigonal-bipyramidal (`@TB1`–`@TB20`), octahedral (`@OH1`–`@OH30`), and allenal (`@AL1`, `@AL2`) centers are accepted and drawn with correct connectivity, but without stereo decoration — their 3D arrangement is not translated into wedges.

Note. Cumulated double bonds always label the central sp carbon (O = C = O, ketenes, allenes): the neighbors are collinear, so without the `C` the two double bonds would read as one long double bond.

6.4 Manual wedges: `!w` and `!h`

For direct control, `!w` draws a solid wedge and `!h` a hashed wedge. These are typed-smiles extensions, not standard SMILES. Like plain bonds, they are colored by the atoms at each end.

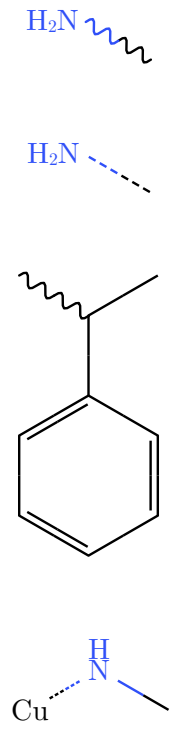
```typst
#smiles("C!wN") \
#smiles("C!hN") \
#smiles("{Nu}!wC({LG|red})=O")
```



6.5 Wavy and dashed bonds: `!s` and `!d`

Two more drawing extensions on single bonds: `!s` draws a wavy (squiggly) bond — the conventional depiction of unspecified stereochemistry or an attachment point — and `!d` draws a dashed bond for hydrogen bonds, partial bonds, and coordination. Like plain bonds, they are colored by the atoms at each end.

```typst
#smiles("C!sN") \
#smiles("C!dN") \
#smiles("CC(!sC)C1=CC=CC=C1") \
#smiles("CN!d[Cu]")
```



Note. `!s` and `!d` carry no cis/trans meaning and never consume a double-bond geometry slot; they can appear next to real `/` and `\` directional bonds.

7 Colors

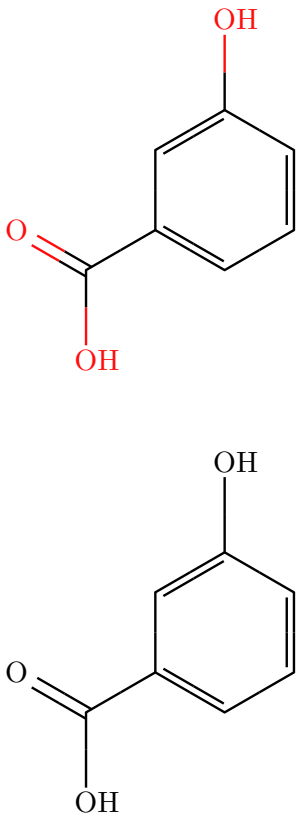
7.1 Jmol CPK palette

Atoms use the Jmol CPK palette by default. Carbon and unlisted elements are black; common heteroatoms get their conventional colors.

N	O	S	P	F	Cl	Br	I

Set `color: false` for a monochrome diagram.

```
#smiles("OC1=CC(=CC=C1)C(=O)O") \
#smiles("OC1=CC(=CC=C1)C(=O)O", color: false)
```



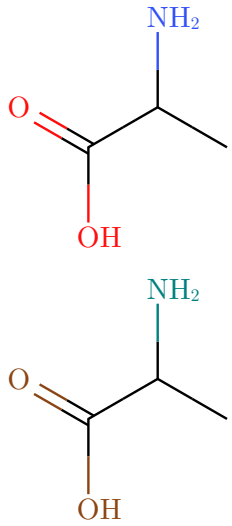
7.2 Custom colors (`atom-colors`)

`atom-colors` is a single dictionary that overrides colors for both elements and arbitrary abbreviated groups. Any Typst color value works — `rgb()`, `luma()`, named constants, or anything else.

Element-symbol keys

Use an element symbol as the key to replace that element's CPK color everywhere it appears, including when referenced as an inline label style (`{OMe|O}` would pick up the overridden oxygen color).

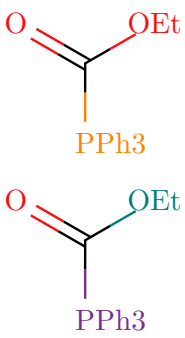
```typst
// default CPK
#smiles("CC(N)C(=O)O")
// oxygen → dark brown; nitrogen → teal
#smiles("CC(N)C(=O)O",
  atom-colors: (O: rgb("#8B4513"), N: rgb("#008080")))
```



Label-name keys

Wrap the label text in braces to target a specific abbreviated group, regardless of any inline style it carries. The key is written as a quoted string because `{` is not a valid Typst identifier character.

```typst
// default — colors come from inline styles or element fallback
#smiles("{>PPh3|P}C({OEt|O})=O")
// override by label name — inline style is ignored for these
#smiles("{>PPh3|P}C({OEt|O})=O",
  atom-colors: ("{PPh3}": rgb("#7B2D8B"), "{OEt}": rgb("#008080")))
```

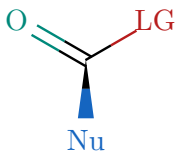


Mixing both key forms

Element keys and label keys can coexist in the same dictionary.

```
// O (element) → teal; {Nu} (label) → navy; {LG} (label) → maroon
#smiles("{Nu}!wC({LG|red})=O",
  atom-colors: (O: rgb("#00897B"), "{Nu}": rgb("#1565C0"), "{LG}": rgb("#B71C1C")))
```

Typst



Priority

Color is resolved in this order for each atom or label:

1. Label-name key in `atom-colors` (e.g. `"{PPh3}"`), if the atom is an abbreviation.
2. Element-symbol key in `atom-colors` (e.g. `O`), for both real atoms and element-style labels.
3. The inline `{label|style}` style from the SMILES string (named color, hex, or element symbol).
4. The default CPK palette.

> **Note.** `color: false` is a hard override that sits above all of the above. When it is set, every atom and label renders in black regardless of any `atom-colors` entries or inline styles. If you want a mostly-monochrome diagram with one or two highlighted groups, keep `color: true` (the default) and put everything else in `atom-colors`.

7.3 Label colors (`{label|style}`)

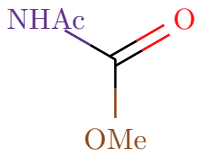
In the SMILES string, write `{label|style}` to color an abbreviated group independently of its element. The style can be a named color, an element symbol, or a hex code.

Named colors:

Name	Swatch	Name	Swatch
red		orange	
yellow		brown	
green		lime	
teal		cyan	
blue		navy	
purple		pink	
black		gray	
silver		maroon	
white			

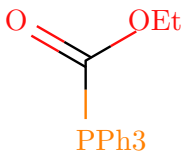
Hex colors: write a `#RRGGBB` hex code after the pipe to use any color. The `#` is just a regular character inside a Typst string literal.

```typst
#smiles("{OMe|#8B4513}C(=O){NHAc|#5B2E8C}")
```



Element symbol: using a symbol as the style applies that element's (possibly overridden) CPK color to the label and its bonds.

```typst
#smiles("{>PPh3|P}C({OEt|O})=O")
```



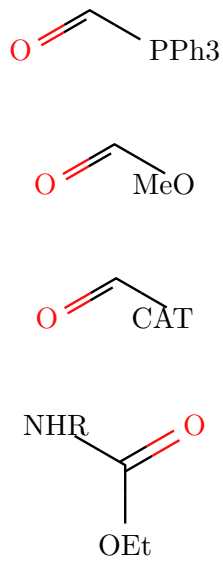
8 Abbreviated groups

8.1 Plain labels

Wrap text in braces `{...}` to place a labeled pseudo-atom that bonds like any other atom. Use `>` inside the label to choose the attachment glyph; the marker is not displayed. Rotate the molecule when needed so the bond approaches the chosen glyph roughly perpendicular to the written label.

```
#smiles("{>PPh3}C=O") \
#smiles("{Me>O}C=O") \
#smiles("{CA>T}C=O") \
#smiles("{OEt}C(=O){NHR}")
```

Typst

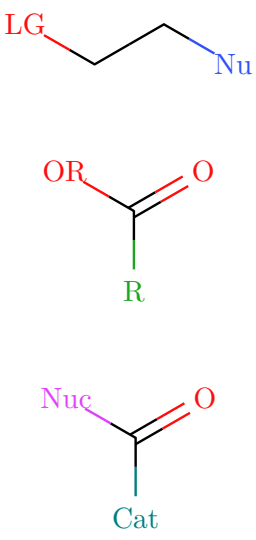


8.2 Colored labels

Add `|style` inside the braces to color a label. Use a named color, an element symbol, or a hex code — see Section 7 for the full list.

```
#smiles("{Nu|blue}CC{LG|red}") \
#smiles("{R|green}C(=O){OR|O}") \
#smiles("{Cat|teal}C(=O){Nuc|#E040FB}")
```

Typst



9 Chemical formulas: `ce()`

For formulas and text equations, `ce` is re-exported from the chemformula package, so one import covers both structures and formulas.

```
#ce("H2SO4") #h(1em) #ce("Ca^2+")
#h(1em) #ce("2 H2 + O2 -> 2 H2O")
```

Typst

$$\mathrm{H_2SO_4} \quad \mathrm{Ca^{2+}} \quad \mathrm{2\,H_2 + O_2 \longrightarrow 2\,H_2O}$$

9.1 Fonts and size

`ce` accepts `font` and `font-size` any other arguments pass through to chemformula.

```
#ce("CuSO4 * 5 H2O") \
#ce("CuSO4 * 5 H2O", font: "Libertinus
Serif", font-size: 13pt) \
#ce("CuSO4 * 5 H2O", font: "PT Sans",
font-size: 13pt)
```

Typst

$$\mathrm{CuSO_4 \cdot 5\,H_2O}$$
$$\mathrm{CuSO_4 \cdot 5\,H_2O}$$
$$\mathrm{CuSO_4 \cdot 5\,H_2O}$$

10 Molecular weight: `mol-weight()`

`mol-weight(smiles)` returns the molecular weight in g/mol as a plain `float`: the sum of IUPAC standard atomic weights over every atom, including implicit and explicit hydrogens. Round it for display with `calc.round`.

```typst
Ethanol: #calc.round(mol-weight("CCO"), digits: 2) g/mol \
Caffeine: #calc.round(
  mol-weight("CN1C=NC2=C1C(=O)N(C(=O)N2C)C"),
  digits: 2,
) g/mol \
Sodium acetate: #calc.round(
  mol-weight("CC(=O)[O-].[Na+]"),
  digits: 2,
) g/mol
```

Typst

Ethanol: 46.07 g/mol
Caffeine: 194.19 g/mol
Sodium acetate: 82.03 g/mol

Dot-separated fragments are summed together, so salts and hydrates work as written. Charges are accepted; as in standard formula-weight arithmetic, the electron mass difference of ions is ignored.

> **Note.** Compilation fails with a descriptive error when the weight is undefined: wildcard `*` atoms, `{label}` abbreviations (no defined composition), and isotope-labeled atoms such as `[2H]` (a specific nuclide needs a nuclide mass, not a standard atomic weight).

11 Reaction schemes

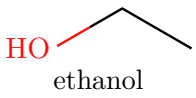
Three helpers compose molecules, formulas, and arrows into schemes.

11.1 mol()

`mol(spec, label: none, offset: (0,0), ..opts)` is a reaction item. `spec` is either any content (`smiles(...)`, `ce(...)`, text) or a SMILES **string**. Passing a string lets `reaction` render the molecule itself so its atoms become addressable by curly arrows (see Reaction mechanisms). String molecules accept common drawing options such as `font-size`, `font`, `bond-stroke`, `color`, `rotation`, `show-all-h`, `lone-pairs`, `atom-colors`, and `show-indices`. `offset` nudges the molecule in bond-length units and switches the reaction into mechanism mode; use `reaction(scale: ...)` to resize a shared mechanism canvas.

```
#reaction(mol(smiles("CCO"),
label: [ethanol]))
```

Typst



ethanol

11.2 rxn-arrow()

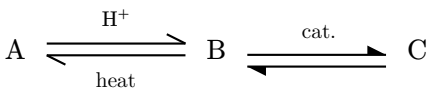
`rxn-arrow(above: none, below: none, dir: "right", kind: "single")` draws an arrow. `dir` may be `"right"`, `"left"`, `"up"`, or `"down"`. `kind` may be `"single"`, `"equilibrium"`, `"equilibrium-filled"`, `"dashed"`, or `"wavy"`. `above` and `below` carry reagents and conditions.

Argument	Default	Effect
`above`	`none`	Label above a horizontal arrow, or right of a vertical arrow.
`below`	`none`	Label below a horizontal arrow, or left of a vertical arrow.
`dir`	`"right"`	Arrow direction: `"right"`, `"left"`, `"up"`, or `"down"`.
`kind`	`"single"`	Arrow style: `"single"`, `"equilibrium"`, `"equilibrium-filled"`, `"dashed"`, or `"wavy"`.

Equilibrium arrows use paired half-heads. The filled variant uses filled half-heads.

```
#reaction(
  ce("A"),
  rxn-arrow(kind: "equilibrium", above: ce("H+"), below: [heat]),
  ce("B"),
  rxn-arrow(kind: "equilibrium-filled", above: [cat.]),
  ce("C"),
)
```

Typst



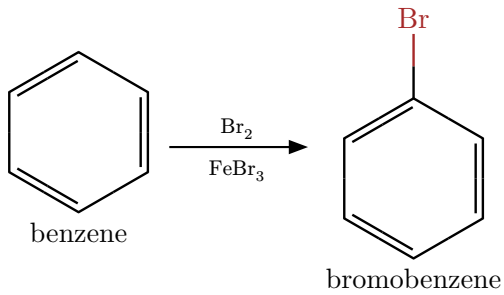
A dashed arrow marks a hypothetical or formal transformation; a wavy arrow marks a distorted or non-elementary one.

```typst
#reaction(
  ce("A"),
  rxn-arrow(kind: "dashed", above: [formal]),
  ce("B"),
  rxn-arrow(kind: "wavy", above: [hv]),
  ce("C"),
)
```

11.3 `reaction()`

`reaction(gap-h: 1.5em, gap-v: 1.5em, scale: 1.0, breakable: false, show-indices: false, ..items)` lays out molecules and arrows left to right. An up or down arrow wraps the scheme onto a new row.

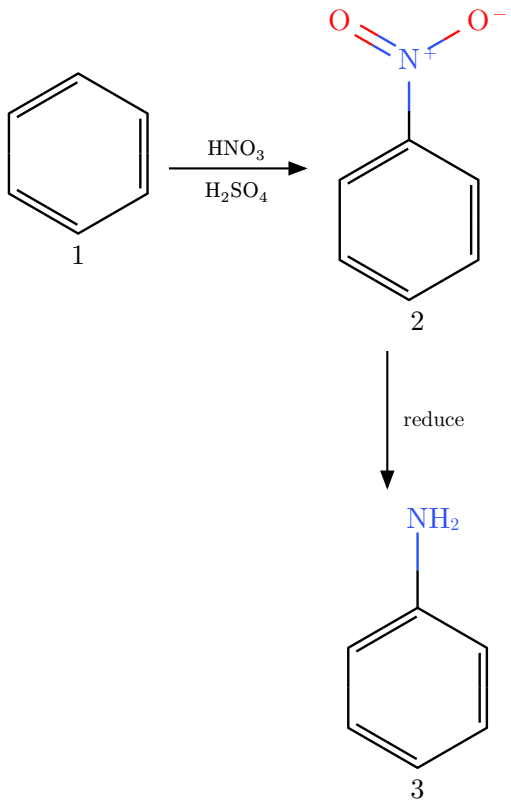
```typst
#reaction(
  mol(smiles("C1=CC=CC=C1"), label: [benzene]),
  rxn-arrow(above: ce("Br2"), below: ce("FeBr3")),
  mol(smiles("BrC1=CC=CC=C1"), label: [bromobenzene]),
)
```



Note. Set `reaction(show-indices: true)` to stamp indices on every string SMILES molecule rendered through `mol("...")` in that reaction. This is useful while authoring large mechanisms with many arrows or highlights. A molecule can still opt out with `mol("...", show-indices: false)`. Molecules already pre-rendered as content, such as `mol(smiles("..."))`, keep their own `smiles()` options.

A wrap-around scheme using a downward arrow:

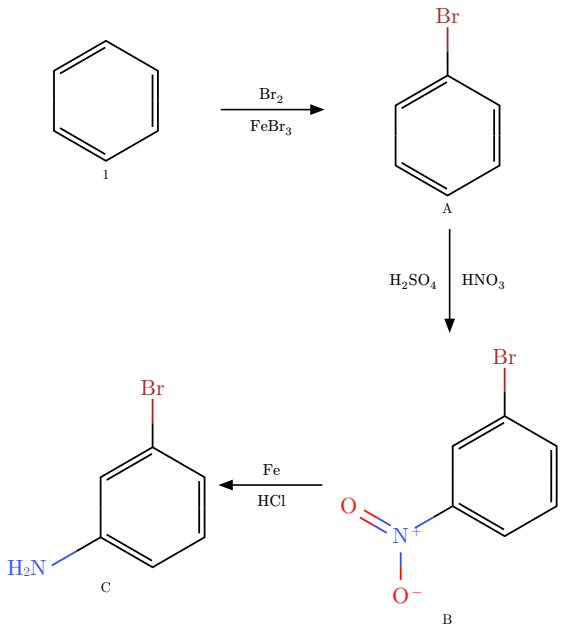
```typst
#reaction(
  mol(smiles("C1=CC=CC=C1"), label: [1]),
  rxn-arrow(above: ce("HNO3"), below: ce("H2SO4")),
  mol(smiles("[O-][N+](=O)C1=CC=CC=C1"), label: [2]),
  rxn-arrow(dir: "down", above: [reduce]),
  mol(smiles("NC1=CC=CC=C1"), label: [3]),
)
```



Uniform scale

Pass `scale` to shrink or enlarge the entire scheme uniformly — molecules, arrows, labels, and separators all change together. This is especially useful for multi-step or wrap-around schemes that would otherwise be too wide.

```typst
// same scheme at three different scales
#reaction(
  scale: 0.75,
  mol(smiles("C1=CC=CC=C1"), label: text(size: 7pt)[1]),
  rxn-arrow(above: ce("Br2"), below: ce("FeBr3")),
  mol(smiles("BrC1=CC=CC=C1"), label: text(size: 7pt)[A]),
  rxn-arrow(dir: "down", above: ce("HNO3"), below: ce("H2SO4")),
  mol(smiles("BrC1=CC(=CC=C1)[N+](=O)[O-]"), label: text(size: 7pt)[B]),
  rxn-arrow(dir: "left", above: ce("Fe"), below: ce("HCl")),
  mol(smiles("BrC1=CC(=CC=C1)N"), label: text(size: 7pt)[C]),
)
```



Note. `reaction(scale: ...)` and `smiles(scale: ...)` are independent. The reaction scale is applied on top of however each individual molecule is sized. To resize everything from a single place, use `reaction(scale: ...)` and let each `smiles()` call use its default.

Page-break behaviour

By default, `reaction` sets `breakable: false`, so the whole scheme moves to the next page as a unit if it does not fit on the current one. This prevents a molecule or vertical branch from stranding on a different page from the rest of the scheme. Set `breakable: true` only for very long schemes that must span pages.

Typst

```typst
// Default — the scheme always stays on one page:
#reaction( … )                      // breakable: false

// Opt in to page splitting for very long schemes:
#reaction(breakable: true, … )
```

12 Reaction mechanisms

`reaction()` also draws electron-pushing mechanisms: curly arrows that flow from a lone pair, bond, or atom into another bond or atom — within one structure or across separate species. It switches from grid layout to a single shared canvas as soon as any curly `arrow()` or `highlight()` is present, or any molecule carries an `offset`. Schemes without these render exactly as before.

12.1 Referencing atoms

Atoms are addressed by their **writing-order index** (0-based), so the SMILES string is never modified. Inside `smiles()` use single-index references; inside `reaction()` prefix them with the species index `s` — the position of the `mol()`/content item in written order (`rxn-arrow`s and annotations are not counted).

Reference	Resolves to
`atom(i)` / `atom(s, i)`	Atom center.
`bond(i, j)` / `bond(s, i, j)`	Midpoint of the bond between atoms i and j.
`lp(i)` / `lp(s, i)`	A lone pair on atom i (use `pair: n` to pick one).
`species(k)`	Bounding-box edge of a whole item (e.g. a `ce()` formula).

Note. Every reference accepts an `offset: (dx, dy)` nudge in bond-length units — the escape hatch for fine-tuning an endpoint or pointing at a single-electron dot. Set `show-indices: true` on a `smiles()` / `mol()` to stamp the indices on the diagram while you write the arrows.

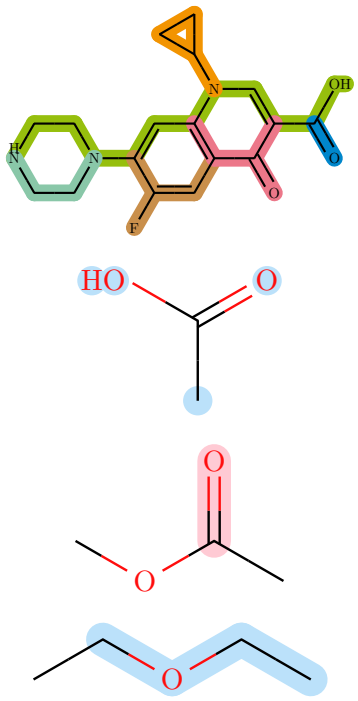
Hydrogen atoms declared inside bracket notation (e.g. `[OH-]`, `[NH4+]`) are also addressable. Each H gets the next available index after all heavy atoms, so `atom(1)` in `[OH-]` refers to the hydrogen. With `show-indices`, the heavy-atom and H badges are centered on their rendered label glyphs. Charge marks are excluded from the atom center, so `atom(0)` in `[O-]` points to the O glyph rather than the combined `O-` label.

12.2 `arrow()` and `highlight()`

`arrow(from:, to:, label: none, color: red, bend: "left", angle: 35deg, half: false)` draws a curly arrow between two references. `bend` is `"left"`, `"right"`, or `none` `angle` sets how strongly it bows; `half: true` draws a fishhook (single-electron) head. `highlight(ref, fill:)` shades an atom (disk) or bond (capsule) behind the structure. Pass an array of references to shade several atoms or bonds with one call. Bond highlights are trimmed away from atom centers by default; set `include-atoms: true` to also shade the endpoint atoms of each highlighted bond. Abbreviated group labels such as `{PPh3}` are highlighted as measured label-width capsules rather than atom-sized disks.

Typst

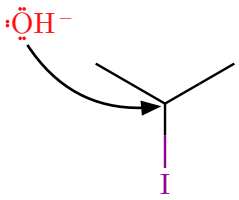
```
#smiles(
  "N1CCN(CC1)C(C(F)=C2)=CC(=C2C4=O)N(C3CC3)
  highlight((bond(0, 5), bond(5, 4),
  bond(4, 3), bond(3, 6), bond(6, 10),
  bond(10, 11), bond(11, 15), bond(15,
  19), bond(19, 20), bond(20, 21),
  bond(21, 23), bond(23, 25)), fill:
  rgb(150, 191, 13), include-atoms:
  true),
  highlight((bond(15, 16), bond(16, 18),
  bond(18, 17), bond(17, 16)), fill:
  rgb(242, 148, 1), include-atoms: true),
  highlight((bond(3, 2), bond(2, 1),
  bond(1, 0)), fill: rgb(137, 199, 168),
  include-atoms: true),
  highlight((bond(6, 7), bond(7, 8),
  bond(7, 9), bond(9, 12)), fill:
  rgb(201, 143, 75), include-atoms:
  true),
  highlight((bond(11, 12), bond(12, 13),
  bond(13, 20), bond(13, 14)), fill:
  rgb(236, 119, 137), include-atoms:
  true),
  highlight((bond(21, 22)), fill: rgb(0,
  134, 203), include-atoms: true),
  color: false,
  rotation: 90deg,
  bond-stroke: 0.8pt,
  scale: 0.5,
)
#smiles(
  "CC(=O)O",
  highlight((atom(0), atom(2), atom(3),
  atom(4)), fill: rgb("#BBE1FA")),
)
#smiles(
  "CC(=O)OC",
  highlight((bond(2, 1)), include-
  atoms:true, fill: rgb("#FFCAD4")),
)
#smiles(
  "CCOCC",
  highlight(
    (bond(0, 1), bond(1, 2), bond(2, 3)),
    fill: rgb("#BBE1FA"),
    include-atoms: true,
  ),
)
```



12.3 A mechanism across species

Hydroxide attacks the central carbon; the C–I bond breaks toward iodide. The nucleophile is offset so the curly arrow reads cleanly.

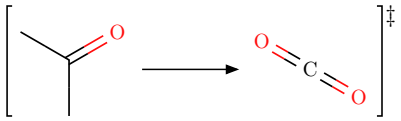
```typst
#reaction(
  mol("[OH-]", lone-pairs: "dots", offset: (1.5, 1)),
  mol("C(I)(C)C"),
  arrow(from: lp(0, 0, offset:(-0.3, -0.2)), to: atom(1, 0, offset : (0.1, -0.1)),
        bend: "right", color : black),
)
```



12.4 `brackets()`

`brackets(body, sup: none, sub: none)` encloses any content in square brackets including another reaction, with optional marks at the corners — a `[‡]` for a transition state or a charge for a reactive intermediate.

```typst
#brackets(
  [#reaction(smiles("CC(=O)C"), rxn-arrow(), smiles("O=C=O"), scale : 0.7)],
  sup: [‡])
```



13 Project-wide defaults

Typst functions support partial application with `.with()`. Calling `smiles.with(...)` returns a new function with the given arguments pre-filled. Rebind the name in your preamble and every subsequent `#smiles(...)` call uses those defaults — including inside `#reaction()`, because the content is evaluated before `reaction` sees it.

Any argument can be pre-filled this way: `bond-length`, `font`, `scale`, `font-size`, `atom-colors`, or any other `smiles` parameter.

Typst

```typst
// ── preamble ──────────────────────────────────────────────────────
#import "@preview/typed-smiles:0.5.0": *

#let smiles = smiles.with(
  bond-length: 0.9,
  font:        "Libertinus Serif",
  atom-colors: (O: rgb("#8B4513"), N: rgb("#008080")),
)

// ── anywhere in the document ──────────────────────────────────────
#smiles("CC(N)C(=O)O")   // uses bond-length 0.9, Libertinus, custom O/N
#reaction(
  smiles("CCO"),
  rxn-arrow(),
  smiles("CC(=O)O"),     // same custom smiles — preamble applies here too
)
```

Note. Any argument passed directly at the call site still overrides the `.with()` default for that call alone.

14 Limitations

- Directional bonds (`/`, `\`) draw the correct cis/trans geometry, but E/Z descriptors are not computed.
- R/S and E/Z descriptors are not calculated.
- Trigonal-bipyramidal (`@TB`), octahedral (`@OH`), and allenal (`@AL`) centers are accepted but drawn without stereo wedges.
- Ring stereochemistry between adjacent centers and bridged bicyclics can overlap or need a manual adjustment (try `rotation`, or the manual `!w` and `!h` wedges). Ordinary substituent branches resolve crowding automatically — a blocking branch flips to the other side of its attachment bond (e.g. the ortho acetyl and carboxyl groups of aspirin), or the colliding bond flips its zigzag turn — always keeping ideal bond angles, so plain branches do not overlap.

15 Quick reference

15.1 Syntax

Syntax	Meaning
`c n o …`	Aromatic atom (lowercase); kekulized on parse.
`.`	Disconnected fragment (no bond): salts, hydrates.
`[...]`	Bracket atom (any element, charges, explicit H).
`@` / `@@`	Tetrahedral anticlockwise / clockwise.
`@SP1`–`@SP3`	Square planar (U / 4 / Z shape), depicted exactly.
`@TB` `@OH` `@AL`	Extended stereo: accepted, drawn without wedges.
`$`	Quadruple bond (four parallel lines).
`/ \`	Double-bond cis/trans geometry.
`!w` / `!h`	Manual solid / hashed wedge (typed-smiles extension).
`!s` / `!d`	Wavy / dashed bond (typed-smiles extension).
`{label}`	Abbreviated group pseudo-atom.
`{>label}`	Abbreviated group anchored at the glyph after `>`.
`{label|style}`	Colored abbreviated group.

15.2 `smiles()` options

Option	Default	Effect
`scale`	`1.0`	Balanced size of everything.
`bond-length`	`none`	Bond length only (`1.0` is 30 pt).
`font-size`	`none`	Atom-label size only.
`font`	`"New Computer Modern"`	Atom-label font.
`bond-stroke`	`none`	Bond width only.
`color`	`true`	CPK colors on or off.
`rotation`	`0deg`	Rotate, labels stay upright.
`mirror`	`none`	Reflect `"horizontal"` or `"vertical"` wedges/hashes swap to keep stereo.
`show-all-h`	`false`	Label carbon hydrogens too.
`lone-pairs`	`none`	Draw lone pairs as `"dots"` or `"lines"`.
`atom-colors`	`(:)`	Color overrides: `O: red` for elements, `"{PPh3}": blue` for labels.
`show-indices`	`false`	Stamp atom indices for writing references.
`..annotations`	—	`arrow()` / `highlight()` items on this molecule.

15.3 `reaction()` options

Option	Default	Effect
`gap-h`	`1.5em`	Horizontal gap between items.
`gap-v`	`1.5em`	Vertical gap between rows.
`scale`	`1.0`	Uniform scale applied to the whole scheme.
`breakable`	`false`	Allow the scheme to split across pages.
`show-indices`	`false`	Default atom-index overlay for string SMILES molecules in this reaction.

15.4 `rxn-arrow()` options

Option	Default	Effect
`above`	`none`	Condition label above a horizontal arrow, or right of a vertical arrow.
`below`	`none`	Condition label below a horizontal arrow, or left of a vertical arrow.
`dir`	`"right"`	Arrow direction: `"right"`, `"left"`, `"up"`, or `"down"`.
`kind`	`"single"`	Arrow style: `"single"`, `"equilibrium"`, `"equilibrium-filled"`, `"dashed"`, or `"wavy"`.

15.5 Data helpers

Helper	Purpose
`mol-weight(smiles)`	Molecular weight in g/mol as a `float` errors on wildcards, abbreviations, and isotopes.

15.6 Mechanism helpers

Helper	Purpose
`mol(spec, label:, offset:, ..opts)`	Reaction item; `spec` as a string is rendered with addressable atoms.
`atom / bond / lp / species`	Atom-index references (optional `offset:`).
`arrow(from:, to:, label:, color:, bend:, angle:, half:)`	Curly electron-pushing arrow.
`highlight(ref, fill:, stroke:, radius:, include-atoms:)`	Shade one atom/bond reference or an array of references.
`brackets(body, sup:, sub:)`	Square brackets with optional corner marks.

15.7 Label color names

Named styles for `{label|style}` — any of these plus any `#RRGGBB` hex:

Name	Swatch	Name	Swatch	Name	Swatch
red		orange		yellow	
brown		green		lime	

teal		cyan		blue	
navy		purple		pink	
black		gray		silver	
maroon		white			